

worklog — Command Cheatsheet

Script lives at: `~/Projects/worklog-cli/worklog.py` Database lives in iCloud Drive, shared across both laptops.

One-time setup (do on BOTH laptops)

```
bash

# 1. Create a folder for the DB in iCloud Drive
mkdir -p ~/Library/Mobile\ Documents/com~apple~CloudDocs/worklog

# 2. Add to ~/.zshrc (edit with: nano ~/.zshrc)
export WORKLOG_DB="$HOME/Library/Mobile Documents/com~apple~CloudDocs/worklog/w
alias worklog="python3 $HOME/Projects/worklog-cli/worklog.py"

# 3. Reload shell config
source ~/.zshrc

# 4. Sanity check (creates the DB on first run)
worklog status
```

Optional model overrides (defaults: qwen2.5:7b, nomic-embed-text):

```
bash

export WORKLOG_CHAT_MODEL="qwen3:8b"
export WORKLOG_EMBED_MODEL="nomic-embed-text"
```

One-time Ollama models (for triage/embed/search):

```
bash

ollama pull nomic-embed-text
ollama pull qwen2.5:7b
```

Daily logging

```
bash
```

```
# Log today's work (the main command – quotes required)
worklog log "Shipped the caching fix on Atlas, 40% faster. Helped Priya debug t

# Log for a specific past date (forgot yesterday?)
worklog log --date 2026-07-09 "Presented Q3 roadmap options to leadership."

# Log multiple times a day – each is a separate entry, totally fine
worklog log "Morning: fixed the flaky test in CI"
worklog log "Afternoon: 1:1 with boss, got go-ahead on the vector search projec
```

Career events

```
bash

# Starting your first (or a new) job – auto-closes the previous company & role
worklog company add "Acme AI" --start 2026-01-05
worklog role add "ML Engineer" --start 2026-01-05

# Promotion – closes current role, opens the new one
worklog promote "Senior ML Engineer" --date 2026-07-01
worklog promote "Staff ML Engineer" --level L6 --date 2028-04-01

# Changing companies (two commands, done)
worklog company add "NewCo" --start 2027-04-01
worklog role add "Senior ML Engineer" --start 2027-04-01

# Leaving a job without a new one lined up
worklog company end "NewCo" --date 2029-06-30

# View your full timeline
worklog career
```

Browsing entries

```
bash

worklog list                                # 30 most recent
worklog list --limit 100                    # more
worklog list --since 2026-04-01 --until 2026-06-30 # a quarter
worklog list --grep caching                 # keyword filter (no AI needed)
```

AI: triage, embed, search (needs Ollama running)

```
bash
```

```
# Extract structured events (wins, collaborations, etc.) from raw entries
worklog triage

# Embed everything new so it's semantically searchable
worklog embed

# Semantic search – meaning, not keywords
worklog search "times I improved performance"
worklog search "led or drove migrations" --top 10
worklog search "helped teammates with data problems"

# Ask a question, get a prose answer synthesized from your entries (RAG)
worklog ask "What roadmap did I present and to whom?"
worklog ask "What did I work on this quarter?" --top 15
worklog ask "When did I last help someone debug?" --sources # show retrieved
```

A good weekly ritual (or make it a cron/launchd job):

```
bash

worklog triage && worklog embed
```

Review season

```
bash

# Markdown report of a period, grouped by type, resume-worthy items starred
worklog export --since 2026-04-01 --until 2026-06-30 > q2_review.md

# Whole tenure at a company
worklog export --since 2026-01-05 --until 2027-03-31 > acme_full.md

# Feed a period to your local LLM for a synthesized self-review
worklog export --since 2026-01-01 --until 2026-06-30 | \
  ollama run qwen2.5:7b "Write my H1 self-review from these events. \
    Group by impact area, lead with the strongest wins, keep metrics."
```

Maintenance

```
bash
```

```
worklog status                                # DB path, current company/role, counts
sqlite3 "$WORKLOG_DB"                        # raw SQL access (edit/fix rows)
# or use DB Browser for SQLite (free GUI) – open the .db from iCloud folder

# Re-run triage from scratch (e.g. after improving the prompt or switching
# chat models). Clears extracted events and marks all raw entries unprocessed.
# Raw entries are never touched – they are the source of truth.
sqlite3 "$WORKLOG_DB" "UPDATE raw_entries SET processed=0; DELETE FROM events;"
worklog triage && worklog embed

# Re-triage only a specific entry (find its id via `worklog list` order or SQL)
sqlite3 "$WORKLOG_DB" "UPDATE raw_entries SET processed=0 WHERE id=3; \
                        DELETE FROM events WHERE raw_entry_id=3;"

worklog triage

# Re-embed everything from scratch (e.g. after changing WORKLOG_EMBED_MODEL –
# embeddings from different models are incompatible, so always do a full reset)
sqlite3 "$WORKLOG_DB" "UPDATE events SET embedding=NULL; \
                        UPDATE raw_entries SET embedding=NULL;"

worklog embed

# Backup (iCloud is sync, not backup – do this occasionally)
cp "$WORKLOG_DB" ~/Projects/worklog-cli/backups/worklog-$(date +%Y%m%d).db
```

Danger zone: deleting entries & resetting tables

These are permanent. Take a backup first (one line, above). Deleted raw entries cannot be re-derived — they ARE the source of truth.

```
bash
```

```

# --- Delete ONE entry completely (raw text + its extracted events) ---
# Find its id first:
sqlite3 "$WORKLOG_DB" "SELECT id, entry_date, substr(raw_text,1,60) FROM raw_en
# Then delete (replace 2 with the id) – events first, then the raw entry:
sqlite3 "$WORKLOG_DB" "DELETE FROM events WHERE raw_entry_id=2; \
                        DELETE FROM raw_entries WHERE id=2;"

# --- Delete all entries in a date range ---
sqlite3 "$WORKLOG_DB" "DELETE FROM events WHERE raw_entry_id IN \
                      (SELECT id FROM raw_entries WHERE entry_date BETWEEN '
                        DELETE FROM raw_entries WHERE entry_date BETWEEN '2026-0

# --- Empty specific tables (structure stays, data goes) ---
sqlite3 "$WORKLOG_DB" "DELETE FROM events;" # extracted even
sqlite3 "$WORKLOG_DB" "DELETE FROM events; DELETE FROM raw_entries;" # ALL jo
sqlite3 "$WORKLOG_DB" "DELETE FROM roles; DELETE FROM companies;" # career

# --- Fix a typo in an entry instead of deleting it ---
sqlite3 "$WORKLOG_DB" "UPDATE raw_entries SET raw_text='corrected text here', p
sqlite3 "$WORKLOG_DB" "DELETE FROM events WHERE raw_entry_id=2;"
worklog triage # re-extracts from the corrected text

# --- Nuke everything and start over ---
rm "$WORKLOG_DB"
worklog status # recreates an empty DB on next run

# --- Reclaim disk space after big deletions (optional) ---
sqlite3 "$WORKLOG_DB" "VACUUM;"

```

Rules of thumb:

- Deleting from `events` is always safe — `triage` can rebuild them from raw entries.
- Deleting from `raw_entries` is forever. When in doubt, edit instead of delete.
- Always delete `events` rows before their parent `raw_entries` row (as shown) so no orphaned events are left pointing at nothing.
- Prefer the GUI for one-off fixes: DB Browser for SQLite → open the .db → Browse Data → right-click a row → Delete Record → Write Changes.

iCloud rules of thumb (important)

1. **One laptop at a time.** SQLite + iCloud is fine for a single user, but never write from both machines simultaneously — iCloud resolves conflicts by duplicating the file, and you'd have to merge by hand.

2. **Let it sync before switching machines.** After logging, give iCloud a moment (check the cloud icon in Finder) before logging from the other laptop.
3. **iCloud may evict the file** to save disk ("Optimize Mac Storage"). If `worklog status` ever says the DB is missing, open the iCloud folder in Finder to force a download, or disable Optimize Storage for reliability.
4. **Backups matter.** Sync propagates mistakes too — a corrupted or mis-edited file syncs everywhere. The `cp` backup above takes 1 second.